

学号： 2106410115

大数据技术与基础 课后作业

学 生 姓 名： 李行健 专 业 班 级： 计算机 2101 班

1.2 编写 Python 程序实现功能：从键盘输入若干姓名，保存在字符串列表中；输入任意姓名，检索列表中是否存在。

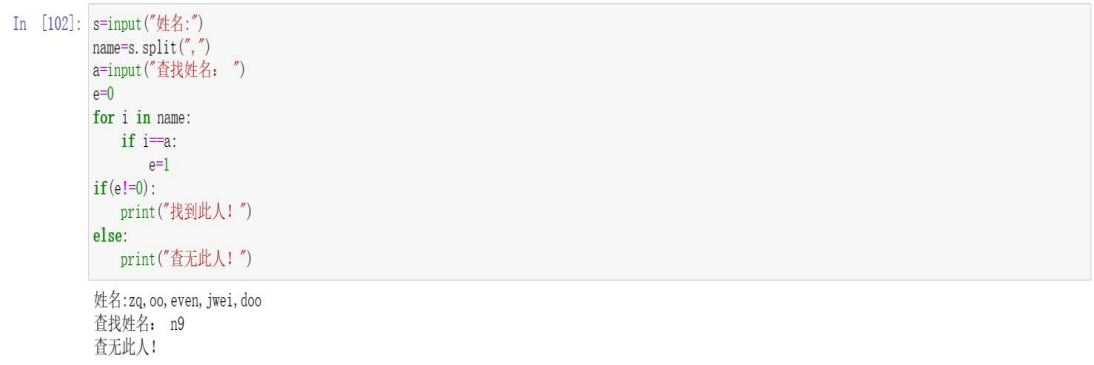
一. 代码

```
s=input("姓名:")
name=s.split(",")
a=input("查找姓名: ")
e=0
for i in name:
    if i==a:
        e=1
if(e!=0):
    print("找到此人!")
else:
    print("查无此人!")
```

二. 程序说明

用字符串的 split 函数将字符串分割成列表，再用 e 作为是否找到人的标志。先设 e 为 0，然后通过遍历姓名，若找到此人，e 变为 1，否则 e 为 0。For 循环结束，通过判断 e 的值来确定找没找到此人。

三. 运行截图



```
In [102]: s=input("姓名:")
name=s.split(",")
a=input("查找姓名: ")
e=0
for i in name:
    if i==a:
        e=1
if(e!=0):
    print("找到此人!")
else:
    print("查无此人!")
```

姓名:zq,oo,even,jwei,doo
查找姓名: n9
查无此人!

2.1 “大润发”、“沃尔玛”、“好德”和“农工商”四个超市都卖苹果、梨、香蕉、桔子、芒果五种水果。使用 NumPy 的 ndarray 实现以下功能。

- (1) 创建 2 个一维数组分别存储超市名称和水果名称；
- (2) 创建 1 个 4×5 的二维数组存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成；
- (3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元；
- (4) “农工商”水果大减价，所有水果价格减少 2 元；
- (5) 统计四个超市苹果和芒果的销售均价；
- (6) 找出桔子价格最贵的超市名称（不是编号）。

一. 代码

- (1) 创建 2 个一维数组分别存储超市名称和水果名称

```
import numpy as np
a1=np.array(['大润发','沃尔玛','好德','农工商']) #创建一维数组 a1 存储超市名称
a2=np.array(['苹果','梨','香蕉','桔子','芒果']) #创建一维数组 a2 存储水果名称
```

(2) 创建 1 个 4×5 的二维数组存储不同超市的水果价格，其中价格由 4 到 10 范围内的随机数生成

```
a3=np.random.randint(4, 11, size=(4, 5))  ##行是超市名称，按 a1 数组排列；  
列是水果名称，按 a2 数组排列  
a3
```

(3) 选择“大润发”的苹果和“好德”的香蕉，并将价格增加 1 元

```
a3[[0, 2], [0, 2]]+=1      ## (0, 0) 为大润发苹果，(2, 2) 为好德香蕉  
a3
```

(4) “农工商”水果大减价，所有水果价格减少 2 元；

```
a3[3, :]-=2      ##第四行均为农工商的水果  
a3
```

(5) 统计四个超市苹果和芒果的销售均价

```
a4=np.mean(a3[:, 0])  ##第一列水果为苹果  
a5=np.mean(a3[:, 4])  ##第五列水果为芒果  
print(a4)  
print(a5)
```

(6) 找出桔子价格最贵的超市名称（不是编号）

```
a6=(a3[:, 3]==(a3[:, 3].max()))  ##第四列水果为桔子  
a1[a6]
```

二. 程序说明

用 np.random.randint 来生成二维数组，用 mean 函数来求平均价格，用 max 来找桔子价格最贵的超市名称

三. 运行截图

```

In [93]: import numpy as np
          a1=np.array(['大润发','沃尔玛','好德','农工商']) #创建一维数组a1存储超市名称
          a2=np.array(['苹果','梨','香蕉','桔子','芒果']) #创建一维数组a2存储水果名称
          print(a1)
          print(a2)

['大润发' '沃尔玛' '好德' '农工商']
['苹果' '梨' '香蕉' '桔子' '芒果']

In [94]: a3=np.random.randint(4,11,size=(4,5)) ##行是超市名称，按a1数组排列；列是水果名称，按a2数组排列
          a3

Out[94]: array([[10,  4,  8,  4,  6],
                [ 7,  8, 10,  7,  4],
                [ 7,  9,  6,  6,  8],
                [ 8,  9,  7,  8,  6]])

In [95]: a3[[0,2],[0,2]]+=1 ## (0, 0)为大润发苹果, (2,2)为好德香蕉
          a3

Out[95]: array([[11,  4,  8,  4,  6],
                [ 7,  8, 10,  7,  4],
                [ 7,  9,  7,  6,  8],
                [ 8,  9,  7,  8,  6]])

In [96]: a3[3,:]=2 ##第四行均为农工商的水果
          a3

Out[96]: array([[11,  4,  8,  4,  6],
                [ 7,  8, 10,  7,  4],
                [ 7,  9,  7,  6,  8],
                [ 6,  7,  5,  6,  4]])

In [97]: a4=np.mean(a3[:,0]) ##第一列水果为苹果
          a5=np.mean(a3[:,4]) ##第五列水果为芒果
          print(a4)
          print(a5)

7.75
5.5

In [101]: a6=(a3[:,3]==(a3[:,3].max())) ##第四列水果为桔子
          a1[a6]

Out[101]: array(['沃尔玛'], dtype='<U3')

```

第三章作业：

三、page 63 3.2

根据银行储户的基本信息，完成以下分析。

- 1) 从“bankpep.csv”文件中读取用户信息。
- 2) 查看储户的总数，以及居住在不同区域的储户数。
- 3) 计算不同性别储户收入的均值和方差。
- 4) 统计接受新业务的储户中各类性别、区域的人数。
- 5) 将存款账户、接受新业务的值转化为数值型。
- 6) 分析收入、存款账户与接收新业务之间的关系。

一. 代码

(1) 从“bankpep.csv”文件中读取用户信息

```

import pandas as pd
user=pd.read_csv('C:\\Users\\kobe\\Desktop\\bankpep.csv')
user

```

(2) 查看储户的总数，以及居住在不同区域的储户数

```
b=0
```

```
for x in user['id']:
```

```
    b+=1
```

```
b ##遍历用户id，从而得到用户个数
```

```
user['region'].value_counts() ##居住在不同区域的储户数
```

(3) 计算不同性别储户收入的均值和方差

```
user.groupby(['sex']).aggregate({'income':['mean','var']}).round(2)
##不同性别储户收入的均值和方差, 保留两位小数
```

(4) 统计接受新业务的储户中各类性别、区域的人数

```
user[['region','sex']].value_counts()
```

(5) 将存款账户、接受新业务的值转化为数值型

```
user[['save_act','pep']]=np.where(user[['save_act','pep']]=='NO',0,1)
user
```

(6) 分析收入、存款账户与接收新业务之间的关系

```
user[['income','save_act','pep']].corr().round(2) ##分析收入、存款账户与接收新业务之间的关系, 保留两位小数
```

二. 程序说明

pd.read_csv 来读入 bankpep.csv, 通过 groupby 对数据分组, 用 for 循环遍历对数据进行统计, 用 corr() 函数对 income, save_act, pep 进行相关性分析

三. 运行截图

```
In [75]: import pandas as pd
user=pd.read_csv('C:\\Users\\kobe\\Desktop\\bankpep.csv')
user

Out[75]:
```

	id	age	sex	region	income	married	children	car	save_act	current_act	mortgage	pep
0	ID12101	48	FEMALE	INNER_CITY	17546.00	NO	1	NO	NO	NO	NO	YES
1	ID12102	40	MALE	TOWN	30085.10	YES	3	YES	NO	YES	YES	NO
2	ID12103	51	FEMALE	INNER_CITY	16575.40	YES	0	YES	YES	YES	NO	NO
3	ID12104	23	FEMALE	TOWN	20375.40	YES	3	NO	NO	YES	NO	NO
4	ID12105	57	FEMALE	RURAL	50576.30	YES	0	NO	YES	NO	NO	NO
...	...	...	...	...	...	...	...	...	...	...	...	...
595	ID12696	61	FEMALE	INNER_CITY	47025.00	NO	2	YES	YES	YES	YES	NO
596	ID12697	30	FEMALE	INNER_CITY	9672.25	YES	0	YES	YES	YES	NO	NO
597	ID12698	31	FEMALE	TOWN	15976.30	YES	0	YES	YES	NO	NO	YES
598	ID12699	29	MALE	INNER_CITY	14711.80	YES	0	NO	YES	NO	YES	NO
599	ID12700	38	MALE	TOWN	26671.60	NO	0	YES	NO	YES	YES	YES

600 rows x 12 columns

```
In [76]: user['region'].value_counts() ##居住在不同区域的储户数

Out[76]:
```

region	count
INNER_CITY	269
TOWN	173
RURAL	96
SUBURBAN	62

Name: region, dtype: int64

```
In [77]: b=0
for x in user['id']:
    b+=1
b ##遍历用户id, 从而得到用户个数

Out[77]: 600

In [78]: user.groupby(['sex']).aggregate({'income':['mean','var']}).round(2) ##不同性别储户收入的均值和方差, 保留两位小数

Out[78]:
```

	income	
	mean	var
sex		
FEMALE	27831.37	1.698137e+08
MALE	27216.69	1.633458e+08

```
In [79]: user[['region', 'sex']].value_counts()
```

```
Out[79]: region sex
INNER_CITY MALE 138
          FEMALE 131
TOWN       FEMALE 92
          MALE 81
RURAL      MALE 49
          FEMALE 47
SUBURBAN   MALE 32
          FEMALE 30
dtype: int64
```

```
In [81]: user[['save_act', 'pep']] = np.where(user[['save_act', 'pep']] == 'NO', 0, 1)
user
```

```
Out[81]:
```

	id	age	sex	region	income	married	children	car	save_act	current_act	mortgage	pep
0	ID12101	48	FEMALE	INNER_CITY	17546.00	NO	1	NO	0	NO	NO	1
1	ID12102	40	MALE	TOWN	30085.10	YES	3	YES	0	YES	YES	0
2	ID12103	51	FEMALE	INNER_CITY	16575.40	YES	0	YES	1	YES	NO	0
3	ID12104	23	FEMALE	TOWN	20375.40	YES	3	NO	0	YES	NO	0
4	ID12105	57	FEMALE	RURAL	50576.30	YES	0	NO	1	NO	NO	0
...	...	...	...	...	...	...	...	...	...	...	...	...
595	ID12696	61	FEMALE	INNER_CITY	47025.00	NO	2	YES	1	YES	YES	0
596	ID12697	30	FEMALE	INNER_CITY	9672.25	YES	0	YES	1	YES	NO	0
597	ID12698	31	FEMALE	TOWN	15976.30	YES	0	YES	1	NO	NO	1
598	ID12699	29	MALE	INNER_CITY	14711.80	YES	0	NO	1	NO	YES	0
599	ID12700	38	MALE	TOWN	26671.60	NO	0	YES	0	YES	YES	1

600 rows x 12 columns

```
In [83]: user[['income', 'save_act', 'pep']].corr().round(2) ##分析收入、存款账户与接收新业务之间的关系，保留两位小数
```

```
Out[83]:
```

	income	save_act	pep
income	1.00	0.27	0.22
save_act	0.27	1.00	-0.07
pep	0.22	-0.07	1.00

四、page 86 可视化综合应用

文件 bankpep.csv 存放着银行储户的基本信息, 请通过绘图对这些客户数据进行探索性分析。

- 1) 客户年龄分布的直方图和密度图
- 2) 客户年龄和收入关系的散点图
- 3) 绘制散点图观察账户(年龄, 收入, 孩子数)之间的关系, 对角线显示直方图
- 4) 按区域展示平均收入的柱状图, 并显示标准差
- 5) 多子图绘制: 账户中性别占比饼图, 有车的性别占比饼图, 按孩子数的账户占比饼图
- 6) 各性别收入的箱须图

一. 代码

(1) 客户年龄分布的直方图和密度图

```
user['age'].plot(kind='hist', bins=10, density=True, title='Customer Age')
user['age'].plot(kind='kde', style='k-')
plt.xlabel('Age')
plt.show()
```

(2) 客户年龄和收入关系的散点图

```
user.plot(kind='scatter', x='age', y='income', title='Customer Income', marker='*', grid=True, xlim=[0, 80], ylim=[0, 60000], label='(age, i
```

```
ncome)')
```

(3) 绘制散点图观察账户(年龄, 收入, 孩子数)之间的关系, 对角线显示直方图
`pd.plotting.scatter_matrix(user[['age', 'income', 'children']], c='b')`

(4) 按区域展示平均收入的柱状图, 并显示标准差

```
mean=user.groupby(['region']).agg({'income':np.mean})
std=user.groupby(['region']).agg({'income':np.std})
mean.plot(kind='bar', yerr=std, color='cadetblue', title='Customer
Income', rot=45)
plt.xlabel('Region')
plt.show()
```

(5) 多子图绘制: 账户中性别占比饼图, 有车的性别占比饼图, 按孩子数的账户占比饼图

```
sex_data=user.groupby(['sex'])['sex'].count()
car_data=user[user['car']=='YES'].groupby(['sex'])['sex'].count()
children_data=user.groupby(['children'])['children'].count()
fig=plt.figure(figsize=(7, 6))
fig.add_subplot(2, 2, 1)
sex_data.plot(kind='pie', title='Customer
Sex', startangle=60, autopct='%1.1f%%')
fig.add_subplot(2, 2, 2)
car_data.plot(kind='pie', title='Customer
Sex', startangle=60, autopct='%1.1f%%')
fig.add_subplot(2, 2, 3)
children_data.plot(kind='pie', title='Customer
Children', startangle=60, autopct='%1.1f%%')
plt.show()
```

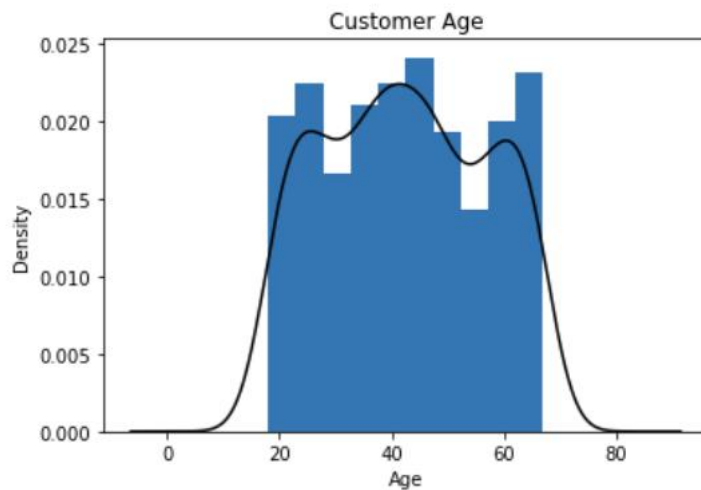
(6) 各性别收入的箱须图

```
user_data=user[['sex', 'income']]
user_data.boxplot(by='sex', figsize=(6, 6))
```

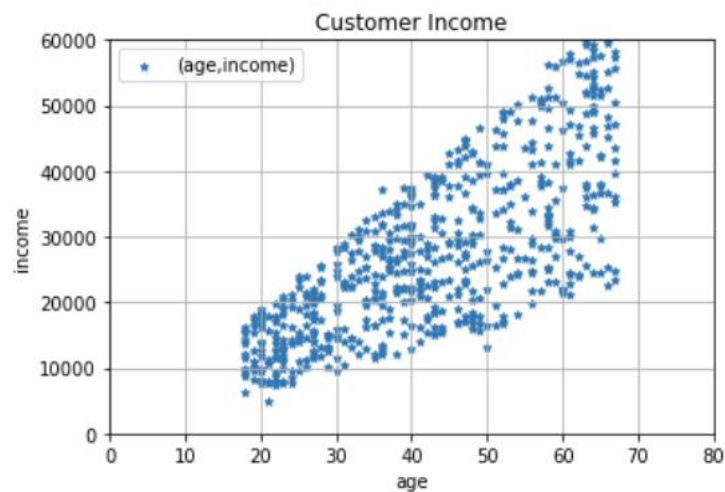
二. 程序说明和运行截图

(1) 客户年龄分布的直方图和密度图

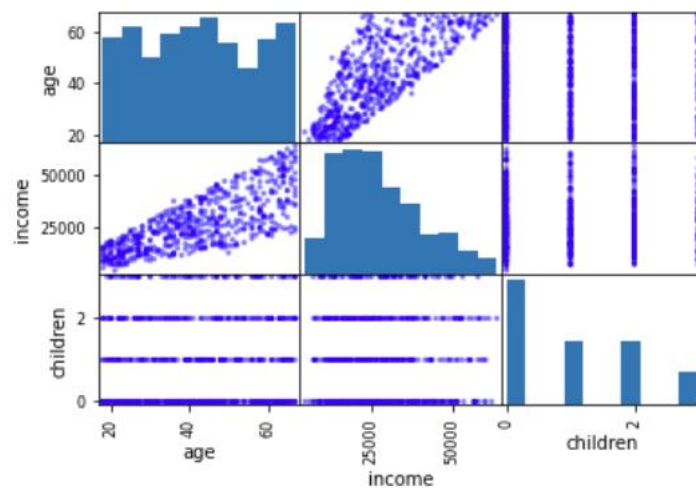
通过 `pd.read_csv` 来读入 `bankpep.csv`, 用 `pd` 的 `plot` 函数来画出直方图和密度图, 参数 `kind` 分别为 `hist` 和 `kde`



(2) 客户年龄和收入关系的散点图
用 `user.plot()` 函数来画散点图，`kind` 参数为 `scatter`，横纵标签分别为 `age`, `income`, 标题 `title` 为 `Customer Income`

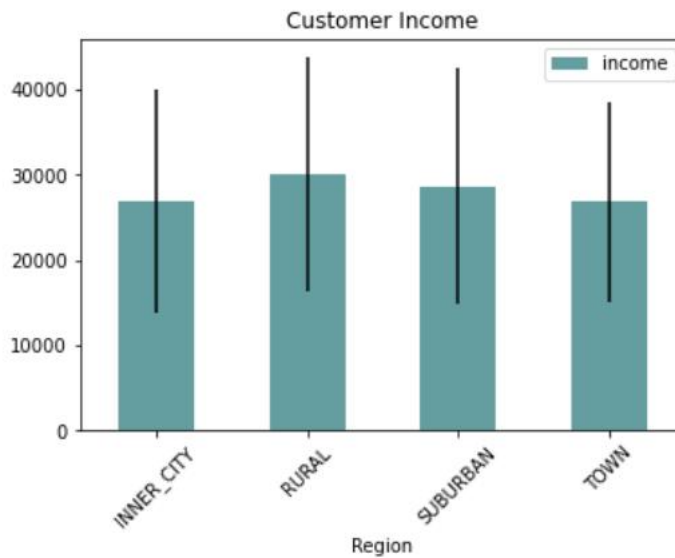


(3) 绘制散点图观察账户(年龄, 收入, 孩子数)之间的关系, 对角线显示直方图
用 `pd.plotting.scatter_matrix()` 函数来生成散点关系图，将 `user` 中的 `age`, `income`, `children` 作为参数画出散点图



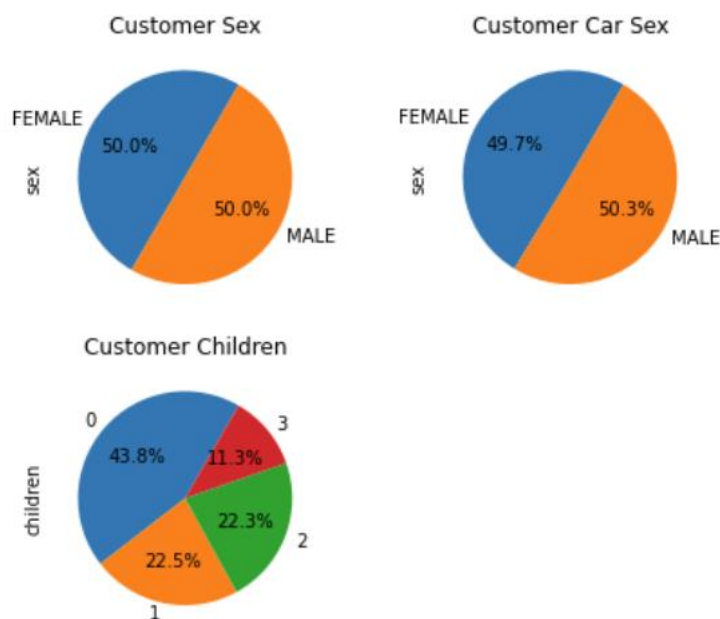
(4) 按区域展示平均收入的柱状图, 并显示标准差

先用 `groupby` 按照区域分组，分别算出平均收入和标准差，使用 `plot` 函数画出柱状图，`kind` 参数为 `bar`



(5) 多子图绘制: 账户中性别占比饼图, 有车的性别占比饼图, 按孩子数的账户占比饼图

先用 `groupby` 和 `count` 分别统计出性别、有车的性别人数、有孩子的人数，然后再用 `plot` 函数画饼图，`kind` 参数为 `pie`，起始角度 `startangle` 为 60



(6) 各性别收入的箱须图

先把 `user[['sex', 'income']]` 赋值给 `user_data` 保存数据，然后用 `user_data.boxplot` 来画箱须图，参数 `figsize = (6, 6)` 来设置长宽

